

Programación II

Segundo Parcial

Profesor: Santiago Gallino.

Importante:

Este trabajo servirá como base para el final (donde se hará un carrito de compras). Es importante tener esto presente a la hora de elegir el tema y contenido que van a utilizar.

Puede, y se recomienda en lo posible, utilizar de base el sitio desarrollado en el primer TP.

Consigna:

Realizar un sitio tipo e-commerce en php de al menos 4 secciones (home, listado de "ítems", detalle de cada ítem, y form de contacto), que cuente además con un panel de administración. El sitio debe implementar inclusión de secciones vía **include/require**, para evitar duplicar HTML. Debe incluir interacción con la base de datos (MySQL/MariaDB), así como programación orientada a objetos. También *debe* utilizar una estructura de HTML5 semántica y correcta, y presentar estilización en CSS con un diseño acorde a la temática abordada (no se aceptarán trabajos en HTML pelado).

La entrega debe cumplir:

Sitio

Como se estipuló, el sitio debe constar de 4 secciones/pantallas (no implica que todas deben figurar en la barra de navegación):

1. "Home": Pantalla introductoria que contenga, al menos, de lo que se trata el sitio, de manera que sea informativo para un nuevo visitante.
2. "Listado": Pantalla donde se listen los "ítems" que se estén ofreciendo, detallando lo más importante. Cada ítem debe poder clickearse para entrar a otra pantalla donde ver más contenido. Estos ítems deben provenir de la información cargada en la base de datos.
3. "Detalle": Pantalla donde se muestren los detalles expandidos de cada ítem.
4. "Contacto": Pantalla con un formulario de contacto conceptualmente integrado al sitio. Esto es, que tenga un uso claro y útil para el usuario según la temática de la página. No es requisito que el form envíe el email.

Panel de Administración

Adicionalmente, debe incluirse una sección aparte que represente un panel de administración de contenidos. Debe contar con lo siguiente:

1. "Login": Pantalla que pida una combinación de email y password para poder ingresar al panel.
2. "ABM (Altas, Bajas, Modificaciones) de Ítems": Pantalla donde se muestre un listado con los "ítems" actuales. Desde acá debe poder accederse al formulario para ingresar un nuevo "ítem", así como acceder a editar o eliminar alguno de los "ítems" existentes.
3. "Formulario de Alta de Ítem": Pantalla que contenga un formulario con los campos necesarios para ingresar un nuevo "ítem" en la base de datos.
4. "Formulario de Edición de Ítem": Pantalla que contenga un formulario con los campos necesarios para editar un "ítem" ya existente. Este formulario debe empezar "pre-poblado" con los datos actuales del "ítem".

Nota: El final va a contemplar "roles" o "niveles" de usuarios, para diferencias usuarios "comunes", que son los que se registran en el sitio

para poder comprar, de los “administradores”, que pueden administrar los contenidos del sitio desde el panel de administración. Esto no aplica para esta entrega. Todos los usuarios van a ser “administradores” por ahora.

HTML+CSS

El código **HTML** debe presentar una estructura **semánticamente correcta**, aprovechando las etiquetas de HTML5 creadas para tal fin. Esto no significa que tiene que haber ejemplos de uso de **todas** las etiquetas que hay, sino que deben usarse las que correspondan para el contenido que se presente. Adicionalmente, **no** significa tampoco que **esté prohibido** el uso de divs, sino que deben utilizarse para estilización o cuando no haya una etiqueta semántica que se ajuste mejor a la situación.

El sitio debe presentar **estilización en CSS**, con un diseño que esté acorde al contenido y target demográfico del sitio. Pueden utilizar frameworks de CSS para ayudarse (como Bootstrap), pero debe incluir estilización personalizada por parte del alumno.

PHP

- El sitio debe implementar una carga de secciones dinámica en un template de base vía parámetros por el “query string” (“GET”), como se vio en la cursada.
- Debe haber uso de programación orientada a objetos. **Como mínimo**, para el “ítem” (ej: producto), usuarios y la conexión a la base de datos.
- Los datos de los “ítems”, así como los usuarios y cualquier otro elemento relevante, deben provenir de una base de datos MySQL o MariaDB. Para interactuar con la base desde php, debe utilizarse la clase nativa **PDO**. Las consultas contra las tablas deben estar dentro de métodos de las clases relevantes, como vimos en cursada con la clase Noticia. **Este punto es indispensable para aprobar el TP.**
- En las consultas que tengan que utilizar datos que provengan del usuario (ej: \$_GET o \$_POST), esos valores *deben* incluirse usando

holders/tokens para prevenir vulnerabilidades de inyección SQL. El uso de dichos valores *dentro* del query **es motivo de desaprobación**.

- **Todas las rutas del TP deben ser relativas, o absolutas calculadas dinámicamente en base a donde se ubique el proyecto.** Es decir, no debe haber **ninguna** ruta que asuma ningún tipo de estructura de archivos/carpetas, ya sea local de la PC del alumno o de algún entorno de desarrollo (ej: Docker, Vagrant).

Ejemplo de ruta correcta: "imagenes/foto.jpg"

Ejemplo de ruta correcta: __DIR__ . "imagenes/foto.jpg"

Ejemplo de ruta **incorrecta**: "C:\Users\santiago\imagenes\mi-foto.jpg"

Ejemplo de ruta **incorrecta**: "http://localhost/davinci/programacion2/tp1/imagenes/mi-foto.jpg"

Base de Datos

- La base de datos debe ser MySQL o MariaDB.
- Como nombre, debe llamarse **dw3_apellido_nombre** (ej: dw3_gallino_santiago) en caso de ser un único integrante, o **dw3_apellido1_apellido2** (ej: dw3_gallino_noto) en caso de ser grupal.
- Debe incluirse el DER de la misma, que permita entender rápidamente su estructura.
- También debe presentarse el archivo SQL, correctamente exportado desde mysqldump o phpmyadmin, para su importación con datos ya cargados.
- Los datos deben ser "reales", no texto simulado ("Lorem ipsum" o similar).
- Debe estar normalizada.
- Debe contar con tablas para:
 - Usuarios.
 - "ítems": que incluya al menos 5 campos.
 - *Al menos* una tabla adicional relacionada con la tabla de "ítems".

Se evaluará y tendrá impacto en la nota también:

- Complejidad de la tarea realizada.
- Prolijidad en el código.
- Uso adecuado de constantes.
- Correcto diseño de la base de datos.
- Buena estructuración del código (ej: organización de carpetas, correcta separación de funcionalidades en funciones, etc).
- Coherencia en los nombres de variables, funciones, etc.
- **Uso correcto de las etiquetas semánticas de HTML.**
- Estilización del sitio.
- Prolijidad en la organización de la carpeta del proyecto.

Modalidad de entrega

La entrega se realizará de manera digital, subiendo un zip/rar con una copia del proyecto completo a la publicación del campus/classroom, con el formato de nombre de archivo:

- En caso de ser 1 miembro: "apellido-nombre.zip" (ej: **gallino-santiago.rar**).
- En caso de ser 2 o más: "apellido1-nombre1_apellido2-nombre2.zip" (ej: **gallino-santiago_noto-federico.rar**).

Ese archivo debe contener:

- Carpeta "sitio" con el sitio completo.
- Carpeta "db" con el archivo SQL para importar.
- Carpeta "der" con el DER de la base de datos.
- Archivo **datos.txt** con todos los datos del estudiante:

Carrera, materia, cuatrimestre, año, turno, comisión, apellido y nombre, docente, carácter de entrega (2do parcial). **Además del email y password del usuario administrador para ingresar al panel.**

El incumplimiento de cualquiera de los requisitos indicados en la modalidad de entrega incurrirá en una penalización de al menos 1 (un) punto menos.

Quedará a discreción del profesor si los exámenes se corrigen en clase o no. También lo hará si es necesario realizar preguntas al alumno como parte de la evaluación, ya sean con respecto a cómo se encaró la entrega, como teóricas.

Si un trabajo se detecta que es una copia de otro, automáticamente recibirá una calificación de 1 (uno). Si además en la misma mesa se presentan copiador y copiado, ambos recibirán esa calificación.